

Aku kasih instruksi masukan dulu jadi jangan dibuat kodingannya terlebih dahulu jadi dashboardnya berfokus pada Okupansi, tunggakan, masa berlaku kontrak. Deliverablenya mencakup Dashboard dinamis + Visualisasi data + Filter & export



Siap, dimengerti! Aku akan *hold* dulu dan tidak menulis kode apa pun sampai instruksimu lengkap.

Pemilihan fokus metrikmu sangat tajam dan tepat sasaran. Untuk sebuah Sistem Pengelolaan Sewa Properti, memantau **Okupansi** (tingkat keterisian ruko), **Tunggakan** (kesehatan finansial/pembayaran), dan **Masa Berlaku Kontrak** (pengingat retensi penyewa) adalah metrik level *enterprise* yang paling dibutuhkan oleh Manager dan Admin.

Aku juga sudah mencatat *deliverables* yang kamu targetkan untuk halaman ini:

- 1 **Dashboard Dinamis:** Data otomatis kalkulasi dari *backend* tanpa perlu *refresh*.
- 2 **Visualisasi Data:** Penggunaan elemen visual (seperti grafik pie atau bar) agar data kompleks lebih mudah dibaca.
- 3 **Filter & Export:** Fitur penyaringan data (misal berdasarkan bulan atau status) dan kemampuan mengunduh laporan (misal ke Excel atau PDF).



Ini struktur databasenya sebagai pengingat



Terima kasih tangkapan layarnya! Diagram relasi entitas (ERD) ini sangat membantu untuk memastikan *query* kita nanti akurat. Strukturnya sudah sangat rapi dan relasi antar tabelnya (terutama **kontrak** sebagai penghubung **tenant** dan **properti**) sudah ideal untuk menyajikan data *dashboard*.

Berdasarkan struktur *database* tersebut, berikut adalah pemetaan cara kita menarik data untuk ketiga metrik utama:

- **Okupansi (Tingkat Keterisian):** Kita akan menghitung jumlah baris di tabel **properti** dan mengelompokkannya (**GROUP BY**) berdasarkan kolom **status_properti** ('tersedia', 'tersewa', 'perbaikan').
- **Tunggakan (Kesehatan Finansial):** Kita akan menyeleksi tabel **pembayaran** . Data yang ditarik

- **Tunggakan (Kesehatan Finansial):** Kita akan menyeleksi tabel `pembayaran`. Data yang ditarik adalah baris di mana kolom `status_bayar` masih 'menunggu' atau 'ditolak', dan `tanggal_jatuh_tempo`-nya lebih kecil (sudah lewat) dari tanggal hari ini. Kita bisa menghitung total nilainya dari selisih `jumlah_tagihan` dan `jumlah_dibayar`.
- **Masa Berlaku Kontrak:** Kita akan fokus pada tabel `kontrak` dengan filter `status_kontrak = 'aktif'`. Kita akan mencari kontrak mana yang selisih antara `tanggal_selesai` dan tanggal hari ini tinggal sedikit (misalnya kurang dari 30 atau 60 hari).

Rekomendasi Arsitektur:

Untuk menghasilkan *dashboard* dinamis yang ringan dan cepat, aku sangat menyarankan kita memproses perhitungan ini di sisi **back-end (Node.js & MySQL)**.

Kita bisa membuat satu *endpoint* API khusus (misalnya `GET /api/dashboard/stats`). Membiarkan MySQL melakukan perhitungan seperti `COUNT` dan `SUM` menggunakan *query* SQL jauh lebih efisien daripada Vanilla JS harus mengunduh ratusan baris data mentah ke *browser* untuk dihitung manual.

Apakah kamu ingin kita mulai dengan menyusun *query* SQL dan *endpoint back-end* untuk *dashboard* ini di `api.js`, atau kamu ingin merancang kerangka UI/UX visualisasinya di `app.js` terlebih dahulu?

aku ada tugas project akhir UAS tentang Sistem Pengelolaan Sewa Properti & Kontrak Tenant yang berbasis sql salah satunya dengan ketentuan entitas CRUD utama Properti, Tenant, Kontrak, Pembayaran jelaskan aku sistemnya seperti apa karna nantinya ada wawancara dari dosen mungkin dari segi penggunaan sintaksnya dan teorinya



Menghadapi wawancara (verbal evaluation) dari dosen untuk project UAS memang butuh persiapan matang, terutama dalam mempertanggungjawabkan logika di balik sistem yang dibangun. Untuk Sistem Pengelolaan Sewa Properti & Kontrak Tenant, dosen biasanya tidak hanya melihat apakah kode atau query-nya berjalan, tetapi *mengapa* kamu mendesainnya seperti itu.

Mari kita bedah sistem ini dari segi teori relasional (ERD) dan praktik sintaks CRUD SQL-nya.

1. Teori & Alur Sistem (Fokus Wawancara Relasi Entitas)

Dalam database relasional, keempat entitas utama ini saling mengikat untuk menjaga integritas data.

Saat wawancara, jelaskan alur ini dengan tegas:

- **Properti (Master Data):** Menyimpan katalog unit yang disewakan (id_properti, nama, tipe, harga_sewa, status_ketersediaan).
- **Tenant (Master Data):** Menyimpan data penyewa (id_tenant, nama, NIK, kontak).
- **Kontrak (Transactional Data):** Ini adalah entitas penghubung (tabel jembatan) antara Properti dan Tenant. Satu tenant bisa menyewa properti, dan satu properti bisa disewa tenant, yang diikat dalam periode waktu tertentu.
- **Pembayaran (Transactional Data):** Terikat langsung dengan Kontrak (bukan langsung ke Tenant atau Properti). Setiap tagihan cicilan atau pelunasan merujuk pada `id_kontrak`.

Pertanyaan Teori Dosen: *"Bagaimana cara kamu mencegah satu properti disewa oleh dua tenant di waktu yang bersamaan?"*

Jawaban: "Melalui validasi di level aplikasi dan database, yaitu dengan mengecek status ketersediaan di tabel `Properti` (misal: 'Tersedia' vs 'Disewa'), serta memastikan tidak ada `Kontrak` aktif yang tumpang tindih untuk `id_properti` yang sama pada rentang tanggal tersebut."

2. Implementasi CRUD & Sintaks SQL

Berikut adalah penjabaran sintaks SQL yang harus kamu pahami luar-dalam.

A. CREATE (Insert Data)

Menambahkan data baru ke dalam sistem.

SQL



```
-- Menambahkan Tenant baru
INSERT INTO Tenant (id_tenant, nama_tenant, nik, kontak)
VALUES ('T001', 'Budi Santoso', '337000111222', '08123456789');

-- Membuat Kontrak Baru (Menghubungkan Tenant & Properti)
INSERT INTO Kontrak (id_kontrak, id_properti, id_tenant, tgl_mulai, tgl_selesai)
VALUES ('K-2604-01', 'P001', 'T001', '2026-05-01', '2027-05-01');
```

Teori untuk Dosen: Urutan `INSERT` sangat penting. Kita tidak bisa membuat `Kontrak` jika `Tenant` dan `Properti` -nya belum terdaftar karena terikat oleh *Foreign Key* (Constraint Constraint).

B. READ (Select & JOIN Data)

Ini adalah bagian krusial. Sistem pengelolaan sewa sangat membutuhkan *dashboard* untuk memantau metrik penting, seperti tingkat hunian (occupancy) atau tenant yang mengalami tunggakan pembayaran (arrears).

SQL



```
-- Menampilkan daftar tenant yang sedang menyewa beserta propertinya
SELECT t.nama_tenant, p.nama_properti, k.tgl_mulai, k.tgl_selesai
FROM Kontrak k
JOIN Tenant t ON k.id_tenant = t.id_tenant
JOIN Properti p ON k.id_properti = p.id_properti
WHERE k.status_kontrak = 'Aktif';

-- Menampilkan riwayat pembayaran untuk suatu kontrak
SELECT id_pembayaran, tgl_bayar, jumlah, status_bayar
FROM Pembayaran
WHERE id_kontrak = 'K-2604-01'
ORDER BY tgl_bayar DESC;
```

Teori untuk Dosen: Jelaskan penggunaan `JOIN` (seperti `INNER JOIN`) untuk menggabungkan tabel agar data yang ditampilkan kepada *user* bermakna (menampilkan nama, bukan sekadar ID angka).

C. UPDATE (Memperbarui Data)

Digunakan saat ada perubahan status. Sangat berbahaya jika dilakukan tanpa `WHERE`.

SQL



```
-- Mengubah status properti saat kontrak selesai
UPDATE Properti
SET status_ketersediaan = 'Tersedia'
WHERE id_properti = 'P001';

-- Mencatat pembayaran yang sudah lunas
UPDATE Pembayaran
SET status_bayar = 'Lunas', tgl_bayar = CURRENT_DATE
WHERE id_pembayaran = 'PAY-001';
```

D. DELETE (Menghapus Data)

Red flag bagi dosen adalah jika kamu menggunakan `DELETE` untuk data transaksi.

SQL



```
-- Menghapus data tenant (Hanya jika benar-benar salah input)
DELETE FROM Tenant
WHERE id_tenant = 'T005';
```

Teori untuk Dosen: Jika ditanya "Bagaimana jika tenant membatalkan sewa, apakah datanya di-DELETE?". Jawablah: "Sebaiknya tidak menggunakan Hard Delete, melainkan Soft Delete." Alih-alih menjalankan perintah `DELETE`, kita melakukan `UPDATE status_kontrak = 'Dibatalkan'`. Ini menjaga rekam jejak finansial dan mencegah *error* pada tabel pembayaran yang bergantung pada data tenant tersebut (*Referential Integrity*).



Fokus Siap Wawancara

- 1 **Hafalkan Relasi Primary Key (PK) & Foreign Key (FK):** Dosen pasti menanyakan di mana letak FK. (Jawaban: FK `id_tenant` dan `id_properti` ada di tabel `Kontrak`).
- 2 **Pahami Normalisasi:** Mengapa data nama penyewa tidak digabung saja di tabel Pembayaran? Agar tidak terjadi redundansi (pengulangan data) jika tenant tersebut memperpanjang kontrak berkali-kali.

```
ini frontend/index.html
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
```



Akhirnya, kepingan *puzzle* terakhir kita telah terpasang dengan sempurna! File `index.html` ini sudah kusimpan.

Dengan selesainya *file* ini, seluruh arsitektur *Frontend* dan *Backend* sistem Manajemen Properti milikmu sudah **100% lengkap**.

Jika `app.js` tadi adalah Operator Lift, dan `views.js` adalah Cetak Biru (Blueprint) ruangnya, maka `index.html` ini adalah **Lahan Tanah Kosong dan Pagar Proyeknya**. Ini adalah tempat pertama di mana seluruh *website*-mu dibangun.

Mari kita bedah sintaksnya dengan bahasa awam:

1. Pemasok Material Bangunan (Bagian `<head>`)

Di bagian atas ini, kamu tidak menulis kode dari nol, melainkan memanggil "pemasok" atau *tools* dari luar negeri (lewat internet/CDN) agar *website*-mu bisa secantik desain *High-Fidelity* di Figma:

- **Tailwind CSS** (`cdn.tailwindcss.com`): Tukang cat kilat. Alih-alih menulis CSS panjang-panjang, alat ini memungkinkanmu mewarnai dan mengatur tata letak hanya dengan menulis `class="bg-blue-600 rounded-xl"` di HTML. Sangat efisien!
- **Phosphor Icons** (`@phosphor-icons/web`): Pemasok ikon estetik. Ini yang membuatmu bisa memunculkan gambar lonceng, rumah, atau avatar hanya dengan mengetik `<i class="ph-bold ph-bell"></i>`.
- **Chart.js** (`chart.js`): Pelukis grafik profesional yang akan bekerja sama dengan `charts.js` milikmu untuk menggambar diagram di *Dashboard*.

2. Panggung Utama (`<div id="app-root">`)

- **Analogi:** Sebuah panggung teater yang masih kosong dan gelap, hanya ada satu tulisan kecil "Menyiapkan sistem...".
- **Penjelasan:** Inilah **jantung** dari *Single Page Application* (SPA). Kamu sengaja membiarkan panggung ini kosong melompong. Kenapa? Karena nanti, si Operator Lift (`app.js`) dan arsitek interior (`views.js`) yang akan memasukkan tembok, kasur, atau meja kerja ke dalam `div app-root` ini secara gaib dan instan sesuai dengan siapa yang sedang *login*.

3. Memanggil Para Tukang Sesuai Urutan (Bagian `<script>`)

Di bagian paling bawah sebelum `</body>` , kamu memanggil semua *file* JavaScript yang sudah kita bedah sebelumnya.

Urutannya sangat fatal dan akurat! Kamu tidak asal menaruhnya:

- 1 `state.js` dipanggil duluan (ingatan harus disiapkan).
- 2 `charts.js` , `auth.js` , `tenant.js` , dan `views.js` dipanggil selanjutnya (semua alat kerja disiapkan).
- 3 `app.js` dipanggil paling terakhir! (Sang Operator baru boleh bekerja kalau semua alat dan ingatan sudah siap).

Kalau urutan ini terbalik (misal `app.js` di atas), aplikasimu akan *error* karena *router* kebingungan mencari halaman yang belum ada.

Kesimpulan & Langkah Selanjutnya

Rin, kamu benar-benar telah menyusun proyek aplikasi web *full-stack* (gabungan *Frontend* & *Backend*) yang solid, modern, dan sangat siap untuk dipresentasikan di depan dosen. Desain antarmukanya responsif, sistem *login*-nya aman, dan struktur *database*-nya saling terhubung (*relational*).

ini folder backend yang terakhir tentang isinya package.json dan package-lock.json

```
{  
  "dependencies": {  
    "bcryptjs": "^3.0.3",
```



Selamat! Ini adalah *file* terakhir dari keseluruhan *backend* kamu. Dengan selesainya bagian ini, kita sudah tuntas membedah 100% sistem *backend*-mu.

Untuk menjelaskan `package.json` dan `package-lock.json` kepada orang awam, mari kita gunakan analogi **Proyek Konstruksi Pembangunan Gedung Apartemen**.

1. Apa itu `package.json` ?

Ini ibarat "**Daftar Belanja Material & Kontrak Kerja**".

File ini memberitahu siapa saja *developer* lain yang aplikasinya ingin dijalankan, alat apa saja yang dibutuhkan agar *server*-mu bisa hidup. Tanpa daftar ini, Node.js tidak tahu harus menginstal apa.

2. Apa itu `package-lock.json` ?

Ini ibarat "Nota Pembelian Resmi beserta Nomor Seri".

Pernahkah kamu membuat *website*, lalu beberapa bulan kemudian rusak karena ada *update* dari pusat? Nah, *file* ini mengunci versi (*lock*) alat yang kamu pakai. Jika kamu menggunakan `express` versi 5.2.1 hari ini, maka besok, bulan depan, atau saat dosenmu mencoba menjalankan kodemu di komputernya, versi yang di-instal pasti persis sama (5.2.1), sehingga terhindar dari *error* perbedaan versi.

Bedah Isi Daftar Belanjamu (Dependencies)

`dependencies` adalah material utama gedungmu. Kalau salah satu dari ini tidak ada, gedungmu (aplikasimu) akan hancur atau tidak bisa berjalan sama sekali:

- 1 `express` : Rangka baja gedungmu. Ini adalah kerangka (*framework*) utama yang membuat komputermu bisa berfungsi sebagai *server web*.
- 2 `mysql2` : Kabel komunikasi dari lobi resepsionis menuju Ruang Arsip Bawah Tanah (Database).
- 3 `cors` : Satpam pemberi Izin Visa. Dia yang memperbolehkan aplikasi *Frontend* (UI) kamu mengambil data dari *Backend*.
- 4 `dotenv` : Brankas tempat kamu menyimpan Buku Catatan Rahasia (file `.env`).
- 5 `bcryptjs` : Mesin pengacak *password*. Saat penyewa mendaftar dengan sandi "12345", mesin ini mengubahnya menjadi teks acak agar tidak bisa dibaca oleh *hacker*.

- 6 `jsonwebtoken` : Mesin pencetak Kartu Akses *Lobby* (Token). Ini yang mengurus sistem login (Satpam Gerbang Utama).
- 7 `multer` : Petugas penerima paket yang khusus mengurus foto, gambar kamar, atau bukti transfer PDF.
- 8 `zod` : Detektor logam di pintu masuk yang memastikan tidak ada formulir data yang salah format (Validasi Input).

Alat Tukang Tambahan (DevDependencies)

`devDependencies` adalah perancah (steger) atau alat bantu proyek. Mereka hanya dipakai saat masa pembangunan, dan tidak akan diikuti saat gedung sudah beroperasi secara resmi (naik ke *hosting*):

- o `nodemon` : Ini adalah **Mandor Robot Pembantumu**. Normalnya, kalau kamu mengubah kodingan, kamu harus mematikan dan menghidupkan *server* lagi secara manual di terminal. Dengan *Nodemon*, setiap kali kamu menekan `Ctrl + S` (Save), si mandor otomatis me-restart *server* secara instan. Sangat menghemat waktu!

kenapa ga dijadikan satu?

Pertanyaan yang sangat bagus! Secara teknis, **sebenarnya bisa saja dijadikan satu file**. Tidak ada aturan hukum pemrograman yang melarangnya.

Namun, alasan saya menyarankan untuk memisahkannya (membuat file khusus untuk logika dashboard) adalah demi **kesehatan jangka panjang** aplikasi Anda. Berikut adalah beberapa alasan utamanya:

1. Menghindari "Spaghetti Code"

Jika semua logika (fungsi API, manipulasi tabel, event listener, dll.) disatukan dalam satu file yang sudah berisi banyak template HTML, file tersebut akan menjadi sangat panjang.

- **Efeknya:** Anda akan kesulitan melakukan *scroll* mencari bagian kode tertentu. Saat Anda ingin memperbaiki *bug*, Anda malah berisiko tidak sengaja menghapus atau mengubah kode UI/HTML-nya.

2. Kemudahan *Debugging* (Mencari Error)

Dengan memisahkan file:

- Jika **tampilan berantakan**, Anda tahu harus membuka `views.js` .
- Jika **data di tabel tidak muncul**, Anda tahu harus membuka `js/dashboard.js` .
- Jika **navigasi atau routing yang salah**, Anda tahu harus membuka `app.js` .
Jika semua jadi satu, setiap kali ada masalah, Anda harus memeriksa ratusan (bahkan ribuan) baris kode untuk mencari di mana letak kesalahannya.

3. Pemisahan Tanggung Jawab (*Separation of Concerns*)

Dalam pengembangan aplikasi profesional, kita membedakan peran:

- **UI/Template (`views.js`)**: Hanya tugasnya "menggambar" HTML.
- **Logika/Bisnis (`dashboard.js`)**: Hanya tugasnya "mengambil data dari server dan mengisi tabel".
Ini membuat kode Anda lebih profesional dan mudah dipahami jika suatu saat Anda ingin menambahkan fitur baru.

4. Performa (Sedikit tapi Penting)

Browser akan memproses file JavaScript dengan lebih efisien jika kodenya terstruktur. Selain itu, jika Anda melakukan perubahan kecil pada logika tabel, *browser* Anda mungkin hanya perlu memuat ulang file JS tersebut, bukan memuat ulang seluruh file UI yang besar.

Apakah Anda tetap ingin menggabungkannya?

Jika proyek Anda skala kecil dan Anda merasa lebih nyaman jika semuanya ada di satu tempat agar mudah dikelola sendiri, **silakan saja**. Tidak ada yang salah dengan itu selama Anda masih bisa menemukan kodenya.

Saran saya: Jika Anda ingin menggabungkannya, Anda bisa menggabungkan `js/dashboard.js` ke dalam `js/views.js`. Namun, pastikan Anda memberikan komentar yang jelas (seperti `// --- LOGIKA DASHBOARD ---`) agar Anda tidak tersesat di tengah kode.

jadi tabelnya dihubungkan dengan database ini dari tenant klik tombol ajukan kan itu udah tuh tercatat di database tinggal admin/manager menyetujui aja gitu sih alur lengkapnya ini Alur Semi-Otomatis (Rekomendasi Terbaik)

Daripada sistem langsung membuat kontrak secara gaib, Admin tetap menjadi penentu akhir dengan memanfaatkan form manual yang sudah kamu buat:

Tenant klik "Ajukan Sewa" di halaman katalog mereka.

Admin melihat pengajuan tersebut di tabel "Daftar Pengajuan Masuk" (berstatus Menunggu).

Admin klik tombol "Setujui".

Begitu diklik, sistem tidak langsung memasukkan data ke tabel kontrak, melainkan me-redirect (mengalihkan) Admin langsung ke halaman form Buat Kontrak yang sudah kamu buat sebelumnya.

Kelebihannya: Saat dialihkan, sistem secara pintar otomatis mengisi (auto-fill) kolom Nama Tenant dan Unit Properti di form tersebut (menggunakan data dari pengajuan tadi).

Admin tinggal mengisi manual data pelengkap lainnya, seperti tanggal mulai sewa, durasi sewa, dan nominal uang muka (DP), lalu klik "Simpan Kontrak".

Cara ini sangat efisien karena kamu tidak perlu membuang form manual yang sudah kamu rancang dari awal, melainkan hanya menjadikannya sebagai kelanjutan dari tombol persetujuan pengajuan. Kalau mau pakai cara ini ngrerombak lagi apa engga membingungkan kodingannya?

Pertanyaan yang sangat bagus. Kabar baiknya: **Tidak, Anda tidak perlu merombak semuanya.**

Anda justru mengambil pendekatan yang sangat cerdas. Anda tidak membuang form "Buat Kontrak" yang sudah ada, Anda hanya menambahkan **jembatan (bridge)** agar form tersebut bisa "menerima data otomatis" dari daftar pengajuan.

Berikut adalah alasan mengapa cara ini **tidak membingungkan** dan **sangat aman** bagi kodingan Anda:

Mengapa Alur Ini Sangat Efisien:

- 1 **Modularitas:** Form "Buat Kontrak" tetap berdiri sendiri. Dia tidak peduli apakah dia diakses secara manual oleh Admin atau otomatis via tombol "Setujui".
- 2 **Zero Overhaul:** Anda tidak perlu merombak form kontrak. Anda hanya perlu menambahkan logika kecil di file JS form tersebut untuk mendeteksi: *"Apakah ada ID pengajuan yang dibawa dari halaman dashboard?"*

- 3 **Data Terintegrasi:** Dengan memanfaatkan `URL Query Parameters` (misalnya `kontrak.html?id_ajukan=123`), sistem akan terasa sangat *seamless* bagi Admin tanpa mengubah satu pun struktur HTML form Anda.

Alur Kerja (Blueprint) yang akan kita terapkan:

- 1 **Halaman Dashboard:** Kita hanya menambahkan fungsi `renderPengajuan()` untuk mengambil data dari database (tabel `pengajuan_sewa`) dan menampilkan tombol "Setujui".
- 2 **Tombol "Setujui":** Tombol ini hanya akan melakukan `window.location.href = 'kontrak.html?id_ajukan=' + id; .`
- 3 **Halaman Form Kontrak (`kontrak.js`):** Kita akan menambahkan fungsi `autoFillData()` yang berjalan saat halaman dimuat. Fungsi ini akan:
 - o Membaca URL untuk mengambil `id_ajukan` .
 - o Fetch data ke API (misal `GET /api/pengajuan/detail?id=123`).
 - o Otomatis memasukkan (inject) data ke input Nama Tenant dan Unit Properti.
- 4 **Tombol "Simpan Kontrak":** Saat tombol ini diklik, Anda cukup menambahkan satu langkah di backend/frontend: mengubah status `status` di tabel `pengajuan_sewa` menjadi `'disetujui'` bersamaan dengan pembuatan kontrak baru.

Kesimpulan:

Struktur kode Anda tidak akan "berantakan" karena kita tidak menghapus atau mengubah logika yang sudah ada; kita hanya **menambah lapisan logika baru** di atasnya.

Jika Anda setuju dengan alur ini (karena ini memang yang paling stabil dan tidak merusak apa yang sudah Anda bangun), saya siap membantu Anda menuliskan:

- 1 **Logika Backend** (API untuk *get* data pengajuan & *set* status).
- 2 **Fungsi JS** untuk tabel dashboard.
- 3 **Fungsi** `autoFill` untuk form kontrak Anda.

Apakah Anda ingin saya buat kodenya sekarang? Kita mulai dari mana dulu?